

Transaction Isolation Levels Cheat Sheet

Matrix of anomalies vs. isolation levels, PostgreSQL defaults, MVCC behavior, and when to use each level.

The Four Isolation Levels (SQL Standard)

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Serialization Anomaly
READ UNCOMMITTED	Possible	Possible	Possible	Possible
READ COMMITTED	Not Possible	Possible	Possible	Possible
REPEATABLE READ	Not Possible	Not Possible	Possible	Possible
SERIALIZABLE	Not Possible	Not Possible	Not Possible	Not Possible

PostgreSQL's Actual Behavior (MVCC makes it stronger than standard)

Isolation Level	Dirty Read	Non-Repeatable Read	Phantom Read	Serialization Anomaly
READ UNCOMMITTED	Not Possible*	Possible	Possible	Possible
READ COMMITTED	Not Possible	Possible	Possible	Possible
REPEATABLE READ	Not Possible	Not Possible	Not Possible**	Possible
SERIALIZABLE	Not Possible	Not Possible	Not Possible	Not Possible

*PostgreSQL treats READ UNCOMMITTED as READ COMMITTED internally. **PostgreSQL's REPEATABLE READ prevents phantom reads (stronger than SQL standard requires).

Anomaly Definitions

Anomaly	Definition	Example
Dirty Read	Read uncommitted data from another transaction	Txn A sees Txn B's INSERT before B commits
Non-Repeatable Read	Same row returns different values when read twice	Txn A reads row, Txn B updates+commits, Txn A reads again → different value
Phantom Read	Same query returns different set of rows when run twice	Txn A counts rows, Txn B inserts+commits, Txn A counts again → different count

Serialization Anomaly	Two transactions produce result impossible in any serial order	Write skew, lost update, read skew in aggregates
------------------------------	--	--

Setting Isolation Level

```
-- Session default
SET default_transaction_isolation = 'repeatable read';

-- Per-transaction
BEGIN;
SET TRANSACTION ISOLATION LEVEL READ COMMITTED; -- default
-- or
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;

-- Check current level
SHOW transaction_isolation;
SELECT current_setting('transaction_isolation');
```

Detailed Per-Level Guide

READ COMMITTED (PostgreSQL Default)

```
BEGIN; -- implicitly READ COMMITTED
SELECT balance FROM accounts WHERE id = 1; -- returns 100
-- Another transaction commits UPDATE SET balance = 50
SELECT balance FROM accounts WHERE id = 1; -- returns 50 (non-repeatable read!)
COMMIT;
```

Behavior:

- Each statement sees committed data as of that statement's start (fresh snapshot per statement)
- Reads never block writes; writes never block reads
- Most concurrent; least isolated

Use when:

- Simple OLTP reads (most web app queries)
- High concurrency is more important than perfect read consistency
- Each query stands alone (no multi-query consistency requirement)

Risks:

- Race conditions if two transactions check then act on the same data
- Classic "check then insert" race: two sessions can both see "no row exists" and both insert

REPEATABLE READ

```
BEGIN TRANSACTION ISOLATION LEVEL REPEATABLE READ;
SELECT balance FROM accounts WHERE id = 1;  -- returns 100 (snapshot taken)
-- Another transaction commits UPDATE SET balance = 50
SELECT balance FROM accounts WHERE id = 1;  -- still returns 100 (same snapshot)
-- Another transaction commits INSERT INTO accounts VALUES (999, 200)
SELECT COUNT(*) FROM accounts;              -- does NOT see the new row either
COMMIT;
```

Behavior:

- Snapshot taken at **first statement** of transaction — all reads see that snapshot
- PostgreSQL REPEATABLE READ also prevents phantom reads (unlike SQL standard)
- UPDATE/DELETE/SELECT FOR UPDATE will still see committed rows — may cause serialization error

Use when:

- Reports that read multiple tables and must see a consistent point-in-time view
- Multi-step calculations where intermediate reads must be consistent
- "Snapshot-isolated" reads for analytics within a transaction

Risks:

- Serialization errors (ERROR: could not serialize access) on concurrent writes
- Application must handle and retry on serialization failure

SERIALIZABLE

```
BEGIN TRANSACTION ISOLATION LEVEL SERIALIZABLE;
-- All transactions execute as if they ran one after another
-- PostgreSQL uses SSI (Serializable Snapshot Isolation) – no locking!
```

Behavior:

- Uses SSI (Serializable Snapshot Isolation) — detects serialization anomalies at commit time
- Transactions that would create impossible serial orderings are **aborted**
- Applications MUST handle ERROR: could not serialize access due to read/write dependencies

Use when:

- Financial transactions (double-spend prevention, transfer operations)
- Any operation where "if I run these transactions one at a time, the result must be correct"
- Constraints that span multiple rows (e.g., "ensure account has sufficient funds")

Risks:

- Higher abort rate under contention (must implement retry logic)
- Small performance overhead (SSI tracking)

```
# Application-level retry pattern (Python example)
for attempt in range(max_retries):
    try:
        with conn.begin():
```

```

        conn.execute("SET TRANSACTION ISOLATION LEVEL SERIALIZABLE")
        # do work
        break # success
except SerializationError:
    if attempt == max_retries - 1:
        raise
    time.sleep(0.1 * (2 ** attempt)) # exponential backoff

```

PostgreSQL MVCC Internals (How Isolation is Implemented)

Each row version has:

xmin = transaction ID that created this version
xmax = transaction ID that deleted/updated this version (0 = visible)

Visibility rule for READ COMMITTED:

A row is visible if:
xmin is committed AND xmin <= current_snapshot_xmax
AND (xmax is 0 OR xmax is aborted OR xmax > current_snapshot_xmax)

Visibility rule for REPEATABLE READ / SERIALIZABLE:

Snapshot taken at transaction start – xmax includes all TXIDs at start
New commits are invisible throughout transaction

Locking Reference

Row-Level Locks

Command	Lock Acquired	Blocks
SELECT	No lock (MVCC)	Nothing
SELECT FOR UPDATE	FOR UPDATE lock	Other FOR UPDATE, FOR SHARE, FOR NO KEY UPDATE
SELECT FOR NO KEY UPDATE	FOR NO KEY UPDATE lock	FOR UPDATE, FOR SHARE
SELECT FOR SHARE	FOR SHARE lock	FOR UPDATE, FOR NO KEY UPDATE
SELECT FOR KEY SHARE	FOR KEY SHARE lock	FOR UPDATE only
UPDATE	FOR NO KEY UPDATE (or FOR UPDATE if PK changes)	
DELETE	FOR UPDATE	

Table-Level Locks (from weakest to strongest)

Lock Mode	Acquired By	Conflicts With
ACCESS SHARE	SELECT	ACCESS EXCLUSIVE only

ROW SHARE	SELECT FOR UPDATE/SHARE	EXCLUSIVE, ACCESS EXCLUSIVE
ROW EXCLUSIVE	INSERT, UPDATE, DELETE	SHARE, SHARE ROW EXCLUSIVE, EXCLUSIVE, ACCESS EXCLUSIVE
SHARE UPDATE EXCLUSIVE	VACUUM, CREATE INDEX CONCURRENTLY	SHARE UPDATE EXCLUSIVE, SHARE, higher
SHARE	CREATE INDEX (non-concurrent)	ROW EXCLUSIVE, higher
SHARE ROW EXCLUSIVE	Rarely used directly	ROW EXCLUSIVE, higher
EXCLUSIVE	Rare	ROW SHARE, higher
ACCESS EXCLUSIVE	ALTER TABLE, TRUNCATE, DROP	Everything

GOTCHA: `ALTER TABLE` takes `ACCESS EXCLUSIVE` lock — blocks ALL reads and writes. Use `ALTER TABLE ... CONCURRENTLY` patterns or `pg_repack` for zero-downtime schema changes.

SKIP LOCKED and NOWAIT

```
-- NOWAIT: fail immediately if can't lock
SELECT * FROM jobs WHERE status = 'pending' LIMIT 1 FOR UPDATE NOWAIT;
-- Raises: ERROR: could not obtain lock on row in relation "jobs"

-- SKIP LOCKED: skip rows already locked (job queue pattern)
SELECT * FROM jobs WHERE status = 'pending' LIMIT 1 FOR UPDATE SKIP LOCKED;
-- Returns next row not locked by another session – perfect for worker queues!
```

Job Queue Pattern:

```
-- Worker 1 and Worker 2 can safely pick different jobs simultaneously
WITH picked AS (
  UPDATE jobs
  SET status = 'processing', started_at = NOW()
  WHERE id = (
    SELECT id FROM jobs WHERE status = 'pending'
    ORDER BY priority DESC, created_at ASC
    LIMIT 1
    FOR UPDATE SKIP LOCKED
  )
  RETURNING *
)
SELECT * FROM picked;
```

Advisory Locks

```

-- Session-level (persist until released or session ends)
SELECT pg_advisory_lock(12345);           -- acquire (blocks if held)
SELECT pg_try_advisory_lock(12345);       -- non-blocking (returns bool)
SELECT pg_advisory_unlock(12345);         -- release

-- Transaction-level (auto-released at transaction end)
SELECT pg_advisory_xact_lock(12345);      -- acquire within transaction
SELECT pg_try_advisory_xact_lock(12345);  -- non-blocking

-- Named locks using hash
SELECT pg_advisory_lock(hashtext('my_process_name'));

-- Use case: ensure only one cron job runs at a time
DO $$
BEGIN
    IF pg_try_advisory_lock(42) THEN
        -- do exclusive work
        PERFORM pg_advisory_unlock(42);
    ELSE
        RAISE NOTICE 'Another instance is running, skipping.';
    END IF;
END $$;

```

SAVEPOINT — Partial Rollback

```

BEGIN;
INSERT INTO orders VALUES (1, 'item_a', 100);
SAVEPOINT after_first_insert;
INSERT INTO orders VALUES (2, 'item_b', 200);
-- Oops, second insert bad:
ROLLBACK TO SAVEPOINT after_first_insert;
-- First insert is still in the transaction
INSERT INTO orders VALUES (2, 'item_c', 300); -- retry with correct data
COMMIT; -- commits item_a and item_c

```

Two-Phase Commit (Distributed Transactions)

```

-- Phase 1: Prepare
BEGIN;
-- do work
PREPARE TRANSACTION 'txn_identifier_123';

-- Phase 2a: Commit
COMMIT PREPARED 'txn_identifier_123';

-- Phase 2b: Rollback
ROLLBACK PREPARED 'txn_identifier_123';

```

```
-- List pending prepared transactions
SELECT * FROM pg_prepared_xacts;
```

Requires `max_prepared_transactions > 0` in `postgresql.conf`.

Isolation Level Decision Guide

Scenario	Recommended Level	Reason
Simple SELECT (no multi-step)	READ COMMITTED	Default, maximum concurrency
Report that reads multiple tables consistently	REPEATABLE READ	Snapshot isolation, no anomalies
Account balance transfer	SERIALIZABLE	Prevent double-spend
Inventory decrement	SERIALIZABLE or explicit FOR UPDATE	Prevent race condition
Batch analytics	REPEATABLE READ	Consistent snapshot for long query
Simple INSERT/UPDATE	READ COMMITTED	No isolation concern
Multi-tenant SaaS (mix of workloads)	READ COMMITTED + explicit locks where needed	Practical balance
Financial ledger operations	SERIALIZABLE	Correctness > performance
OLAP reporting queries	REPEATABLE READ	Consistent long-running reads

Quick Reference Card

DEFAULT in PostgreSQL: READ COMMITTED

When do I need higher isolation?

- └ I read multiple rows/tables and need consistent snapshot → REPEATABLE READ
- └ I read then conditionally write based on what I read → SERIALIZABLE
- └ Financial or inventory (can't have race conditions) → SERIALIZABLE
- └ I only read once, no dependency on previous reads → READ COMMITTED (default)

SERIALIZABLE has overhead:

- Must handle serialization errors (retry logic required)
- Small CPU overhead for SSI tracking
- Under high contention, abort rate increases

MVCC summary:

- Readers never block writers
- Writers never block readers

- Each transaction sees a snapshot of committed data
- Dead row versions accumulate → autovacuum cleans them up